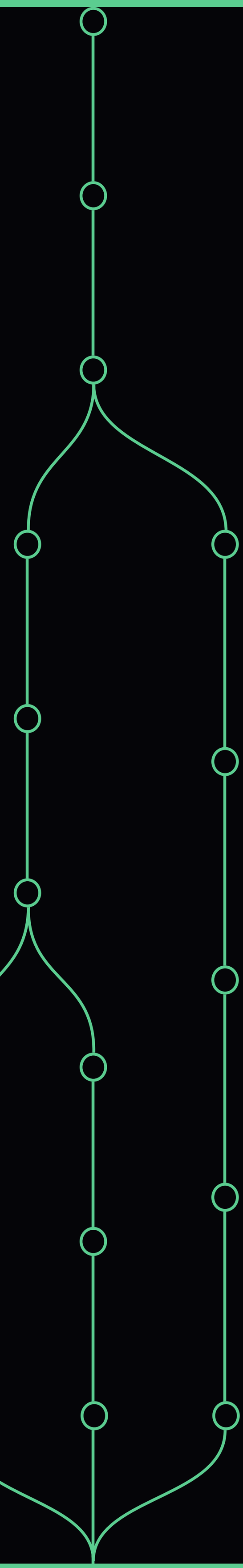
Usof frontend

Track Full Stack

August 31, 2026



Innovation Campus

ucode connect

Contents

Engage 2

Investigate 3

Act: Basic 5

Act: Creative 7

Document 8

Share 10

Innovation Campus

ucode connect

Engage

DESCRIPTION

Hi, developer!

Building high-quality frontend is often quite challenging. A good interface is born at the junction of many technologies and approaches. It is an aesthetic design built on the preferences of the target audience and a call to action, a well thought-out interface and an understanding of the specifics of interaction with the UI/UX software product.

Functionality should serve customer needs and meet customer expectations. When it comes to features, more does not necessarily mean better. Good web design has no features that customers could find annoying. A product with fewer features that do exactly what the user needs can be perceived as higher quality than a product packed with features, some of which do not work properly or have little real value.

To offer good user experience, the web page should be obvious. Users shouldn't have any questions or misunderstandings: every element on the page must work as expected. So, when developing an interface, it's best to work on the principle of "keeping it simple".

Moreover, the various functionalities of a website should work fast, not overload the browser and, of course, not generate any errors. To create a high-quality website, you need to consider the number of potential visitors - this will help you prevent overloads. The developer should handle all possible vulnerabilities on both the server and the client.

Use the latest technologies to withstand real-world conditions over time. This way, your web pages will be available to users on various devices, platforms, and versions of different browsers.

BIG IDEA

Interface for an API.

ESSENTIAL QUESTION

How is creating the frontend side for an API different from writing a regular website's frontend?

CHALLENGE

Develop an interface as the second step in creating your website.

Investigate

GUIDING QUESTIONS

We invite you to find answers to the following questions. By researching and answering them, you will gain the knowledge necessary to complete the challenge. To find solutions, ask the students around you and search the internet. We encourage you to ask as many questions as possible. Note down your findings and discuss them with your peers.

- What is the difference between single-page and multi-page apps?
- Why is website security important?
- What kinds of website vulnerabilities and threats are there?
- How to protect your website from hacker attacks? What can be done on the frontend side in this regard?
- How does the backend interact with the frontend?
- Why do people prefer using frontend frameworks instead of writing pure HTML/CSS/JS code?
- What frameworks and libraries are useful for layouts?
- Why does the page load speed matter?

GUIDING ACTIVITIES

Complete the following activities. Don't forget that you have limited time to overcome the challenge. Use it wisely. Distribute tasks correctly.

- Read the story.
- Clone your git repository issued on the challenge page.
- Explore new things. Talk, discuss and communicate.
- Investigate which tasks take the most time during development.
- Investigate why planning may help you to save time and make error-free web services.
- Compare the frameworks you know. What advantages does React have over others?
- Find some tips that may help you in developing this app.
- Install npm and the latest version of React.
- Create a new project.
- Run the app to get the default homepage opened in your browser. Complete the configuration.
- Create the smallest React example - a heading saying "Hello, world!" on the page.
- To better understand the basics of website design, check out The design of everyday things by Don Norman.
- Start to develop the solution. Strive for excellence. Test your code.

ANALYSIS

Analyze your findings. What conclusions have you made after completing guiding questions and activities? In addition to your thoughts and conclusions, here are some more analysis results.

- Be attentive to all statements of the story.
- Perform only those tasks that are given in the story.
- Analyze all information you have collected during the preparation stages.
- Complete the Document section while developing a challenge. It is described after the Act section.
- You can proceed to Act: Creative only after you have completed all requirements in Act: Basic. But before you begin to complete the challenge, pay attention to the program's architecture. Take into account the fact that many features indicated in the Act: Creative require special architecture. And in order not to rewrite all the code somewhen, we recommend you initially determine what exactly you will do in the future. Note that the Act: Basic part gives the minimum points to validate the challenge.
- Perform the challenge in HTML, CSS, JavaScript with React framework .
- Submit all necessary files to your repository with one of structures to the web directory.
- For the assessment you will need the API from usof backend . Make sure that the latter is at your repository in API directory and it starts easily.
- If you use a package manager, package.json must be at the repository root.
- Usage of third-party frameworks is considered a cheat, and your challenge will be failed.
- Be sure to validate all input fields with relevant error messages.
- Your web app must be responsive and correctly displayed on different screen resolutions (mobile, tablet, etc.).
- Pages must represent all actual data from API.
- Your web app must run locally.
- Complete tasks according to the rules specified in the following style guides:
    - HTML and CSS: Google HTML/CSS Style Guide. As per section 3.1.7 Optional Tags, it doesn't apply. Do not omit optional tags, such as `<head>` or `<body>`
    - JavaScript:
        - JavaScript Style Guide and Coding Conventions
        - JavaScript Best Practices
        - Airbnb React/JSX Style Guide
- The solution will be checked and graded by students like you. Peer-to-Peer learning.
- If you have any questions or don't understand something, ask other students, Google it or ask AI chatbot like ChatGPT, Claude, Gemini. However, remember to verify the information you receive.
- AI tools (e.g., ChatGPT, Claude, Gemini, or similar) may be used as learning assistants to ask questions, explore concepts, understand documentation, debug errors, and receive guidance. They should support your learning process but not replace it. Generating complete assignment solutions or ready-to-submit code with AI is prohibited. Your final submission must be written and understood by you. Remember that you have ownership of, accountability for, and responsibility for the code you commit.

Act: Basic

ALLOWED STACK

`HTML, CSS, React framework, Redux`

DESCRIPTION

Create a web app based on the API from the challenge usof backend .
That means that now you must create the visualization and implement full functionality for every endpoint from the previous challenge.

Now you need to learn and use the React framework.
React is a library, created by Facebook and a community of developers, that allows for integration with many interesting components.
In fact, developers can create their own components and make them accessible to the community.

Everything you need to know about React can be reduced to five key concepts:

- components
- JSX
- props and state
- the component API
- component types

These are only the major concepts and getting to know them won't be enough to become a React master.

But it is enough for the purposes of this app (unless you decide to build something on the scale of Facebook, of course). We also advise you to get acquainted with React code best practices.
This challenge is mostly creative, so there aren't many strict rules.

Think outside the box and create an original product!

And remember: the more attractive the registration form is, the more users will want to sign up for your app.

Every page must contain a header and a menu bar.
The header must have the following elements:

- name of your service
- a search bar for fast navigation across the site
- the role of the current user, login and avatar

You can add the menu bar to the header or implement it in any other way.
However, it must be present.

If the user is not signed up, you can display some default values for login and avatar. Personal image is also a button leading to the personal profile page.
A logged in user must have an opportunity to log out on every page of your service.

Implement a user profile page and a main page.
On the main page of your site, display recently added posts sorted by last added.

The user can manage sorting and filtering parameters.
Show the user's posts on the user profile page.
Display posts once the user visits the page, or opens a separate tab.

An authorized user can:

- visit profile page and edit some personal data
- change some data of their own posts (see your previous challenge)
- choose the category of posts to display
- sort their own posts
- choose the parameters for sorting

It is implied that your service will have a lot of users and publications.
Correspondingly, you must implement content pagination.
It can be endless scrolling or pagination with the definition of a specific page.
Search for pagination types and decide which one is more compatible with your needs.

Every post must have a preview. Think about the preview size for its proper representation in accordance with your pagination parameters. These parameters must be easily changeable. Post visualization has to display all of the fields from the corresponding endpoint (publish date, title, author, etc.). Nesting of your threads must be logical - so that it's clear to which posts the comments belong.

Moreover, you must implement a possibility for users to comment comments.

By default, comments of a post must be sorted in ascending order by the number of likes. It's up to you whether to include posts' most liked comments in the preview. Keep in mind that your interface must be user-friendly.

SEE ALSO

- Framework:
  - React
  - React-Redux
- Middleware:
  - Redux DevTools
  - Fetch API
  - axios
  - Redux Thunk
- Package managers:
  - npm
  - yarn
- Boilerplate:
  - Wiki
  - yarn
- Other:
  - Flexbox
  - TinyMCE

Act: Creative

DESCRIPTION

It is the place where your imagination and creativity plays a major role. Implement additional features to make the program better and more unique. Listed below are a few ideas you can add to your program. You can come up with everything you want to improve your program. Creative features:

- a button for sharing the specified post on different social media such as Facebook or Instagram
- different types of reactions for posts and comments (think about how it will affect user rating)
- a footer (general for the whole website)
- a page with statistics of visits, etc.
- other useful and interesting features

But the main focus is the appearance of your service, so the uniqueness of design will play a significant role in your grade.

Document

DOCUMENTATION

One of the attributes of Challenge Based Learning is documentation of the learning experience from challenge to solution. Throughout the challenge, you document your work using text and images and reflect on the process. These artifacts are helpful for ongoing reflection, informative assessment, evidence of learning, portfolios, and telling the story of challenge. The end of each phase (Engage, Investigate, Act) of the challenge offers an opportunity to document the process.

Much of the most profound learning occurs by considering the process, thinking about one's own learning, analyzing ongoing relationships with the content and between concepts, interacting with other people, and developing a solution. During learning, documentation of all processes will help you analyze your work, approaches, thoughts, implementation options, code, etc. In the future, this will help you understand your mistakes, improve your work, and read the code.

It is vital to understand and do this at the learning stage, as this is one of the skills you will need in your future job. Naturally, the documentation should not be voluminous. It should be presented in an accessible, logical, and connected form.

So what should you do?

- A nice-looking and helpful README file. For people to want to use your product, their first introduction must be through the README on the project's git page. Your README file must contain:
  - Short description. It means that there must be some info about what your project is. For example, what your program does.
  - Screenshots of your solution. This point is about screenshots of your project "in use".
  - Requirements and dependencies. A list of everything that needs to be installed on any machine to build your project.
  - How to run your solution. Describe the steps from cloning your repository to the first launch of your program.
- Full-fledged documentation in any form is convenient for you. By writing this, you will get some benefits:
  - you have an opportunity to think through implementation without the overhead of changing code every time you change your mind about how something should be organized. You will have excellent documentation available for you to know what you need to implement
  - if you work with a development team and they have access to this information before you complete the project, they can confidently start working on another part of projects that will interact with your code
  - everyone can find how your project works
- Your documentation must contain:
  - description of progress after every competed CBL stage
  - description of the algorithm of your whole program

Keep in mind that peers will check the implementation of this stage at the assessment!

Also, several links can help you:

- Make a README
- How to write a readme.md file?
- A Beginners Guide to writing a README
- Google Tools - a good way to journal your phases and processes:
  - Google Docs
  - Google Sheets
- Git Wiki - a section for hosting documentation on Git-repository
- Haroopad - a markdown enabled document processor for creating web-friendly documents
- Canva - a good way to visualize your data
- code commenting - source code clarification method. The programming language determines the syntax of comments
- and others to your taste

Share

DESCRIPTION

Last but not least, the final stage of your work is to publish it on LinkedIn in a form of post. This step isn't just about showcasing your work — it's a crucial part of Challenge Based Learning framework. During this stage, you will discover ways of getting external evaluation and feedback on your work. Analyzing your process helps you to understand your strengths and areas for improvement, while sharing your insights invites valuable feedback from peers and professionals.

Pay attention to details:

- aim to make your post both informative and reflective
- start by briefly introducing the challenge and its purpose
- highlight the key challenges you faced and the technologies or solutions you used to overcome them
- share any significant findings or lessons learned during the process
- conclude with a reflection on how this experience has prepared you for future work and invite others to share their thoughts

Helpful tools:

- Canva - a good way to visualize your data
- QuickTime or OBS - an easy way to capture your screen, record video or audio

Don’t forget to tag your post with #InnovationCampusKhPI to connect with the community! Follow and mention via @ Innovation Campus of NTU "KhPI"!